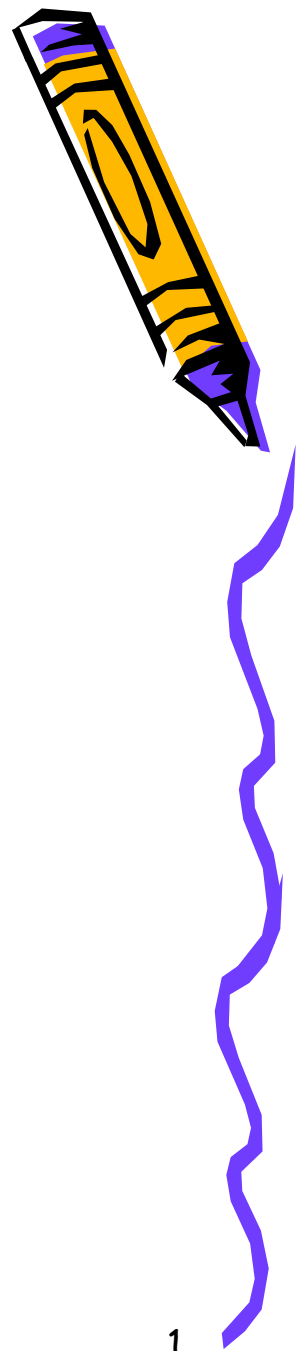


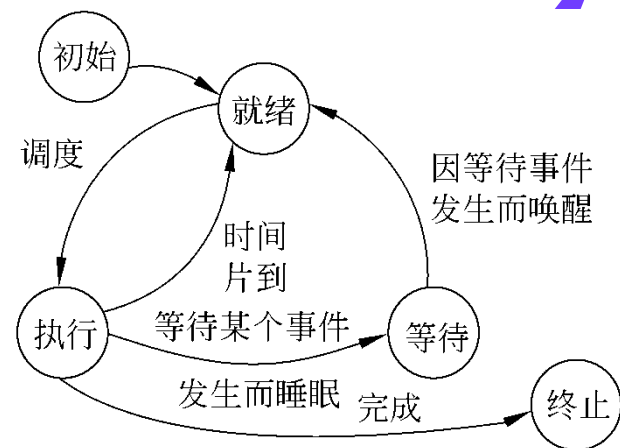
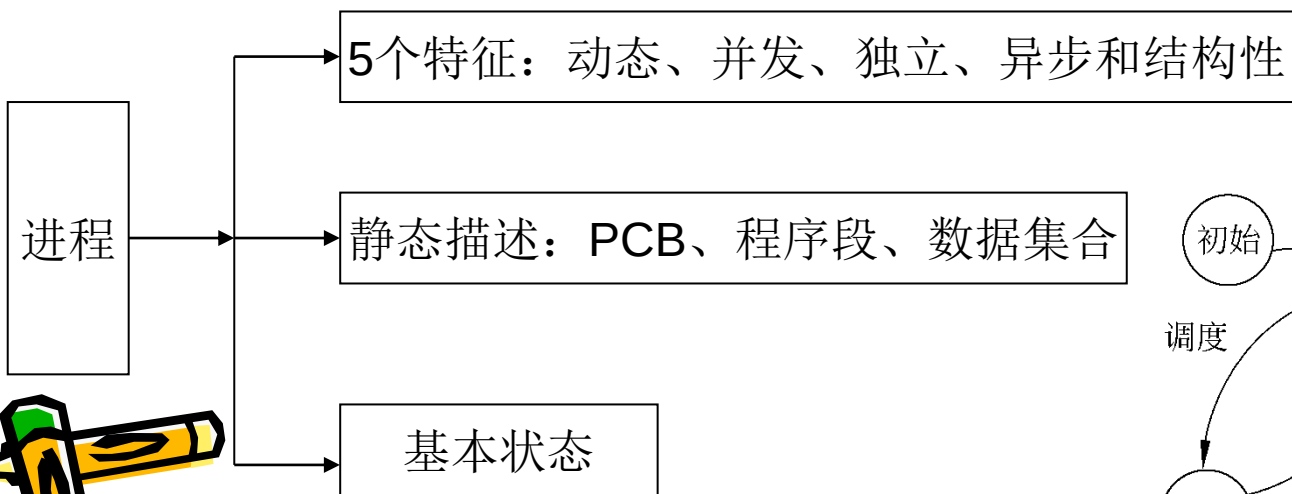
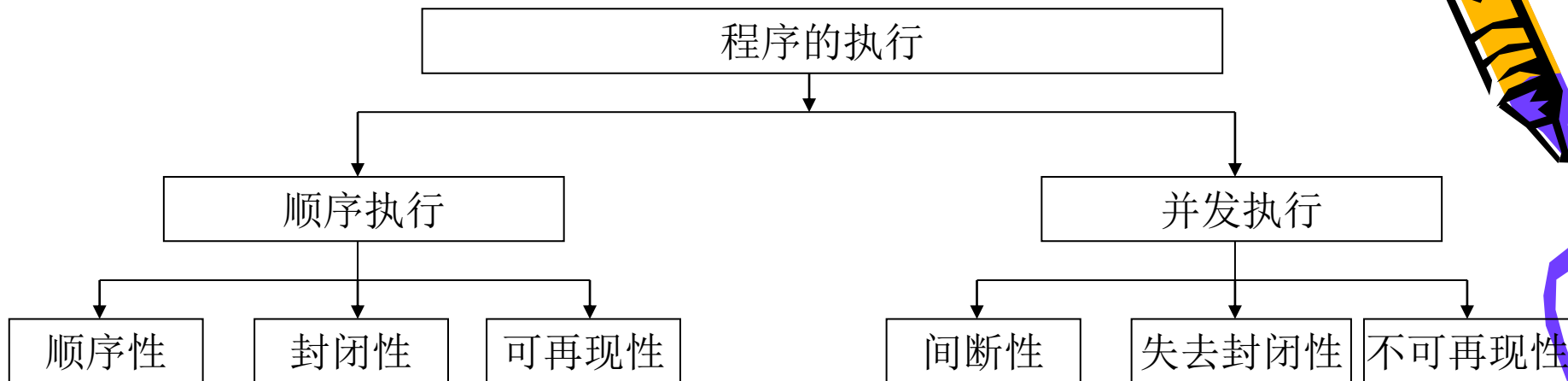
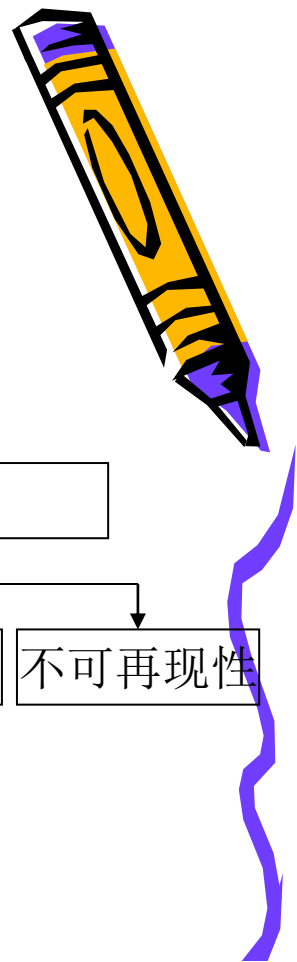
内容提要

- 进程的描述和控制
- 进程的同步与通信
- 调度与死锁



2021-04-22

进程的描述与控制



2021-04-22



习题1

- 进程与线程有什么区别？



比较的角度：调度、
并发性、拥有资源、
系统开销

习题1

- 进程与线程有什么区别？

答：

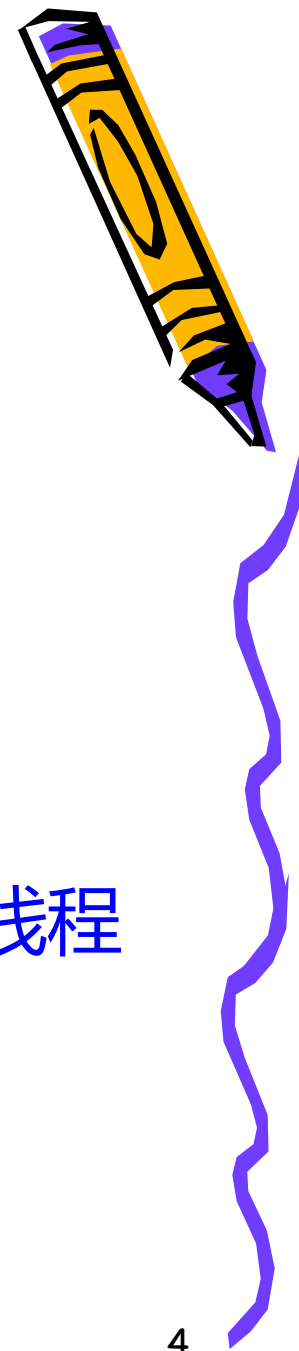
(1) 调度

在传统操作系统中，调度和分派的单位是进程

在引入线程的操作系统中，调度和分派的单位是线程



2021-04-22



习题1

- 进程与线程有什么区别？

答：

(2) 并行性

在引入线程之间操作系统中，不仅进程之间可以并发执行，而且一个进程的多个线程之间也可以并发执行，从而可以更为有效地使用资源，并提高系统的吞吐量



习题1

- 进程与线程有什么区别？

答：

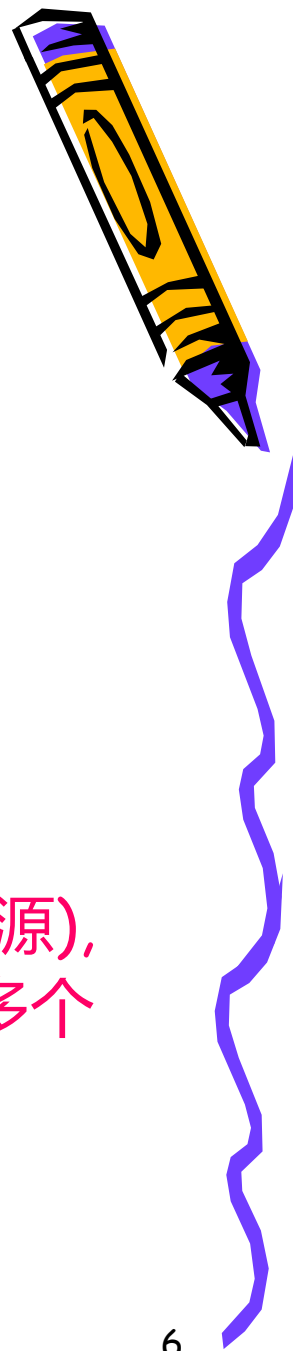
(3) 拥有资源

进程是拥有资源的基本单位

线程自己不拥有系统资源(也有一点必不可少的资源),
但它可以访问其隶属进程的资源, 同一进程中的多个
线程共享其资源



2021-04-22



习题1

- 进程与线程有什么区别？

答：

(4) 系统开销

由于在创建,撤销或切换进程时,系统都要为之分配或回收资源,保存CPU现场.因此,操作系统所付出的开销将显著地大于在创建,撤销或切换线程时的开销.



习题2

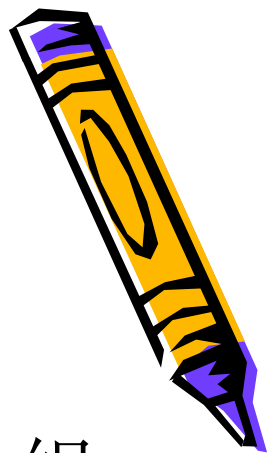


- 通常，进程实体是由_____，
_____和_____这三部分组成，其中_____是进程的唯一标志。

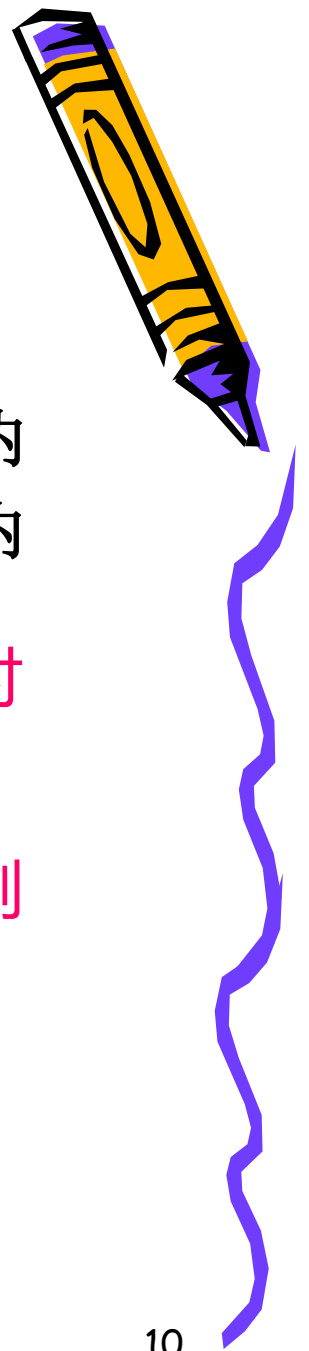


习题2

- 通常，进程实体是由程序段，数据段和PCB这三部分组成，其中PCB是进程的唯一标志。



习题3



- 并发性是指若干事件在（ ）发生。
- **A.**同一时刻
- **B.**同一时间间隔内
- **C.**不同时刻
- **D.**不同时间间隔内

并发性 (concurrency) 是指2 个或多个事件在同一时间间隔内发生

并行性 (parallel) 是指两个或者多个事件在同一时刻发生



习题4

(程序并发执行的特性)

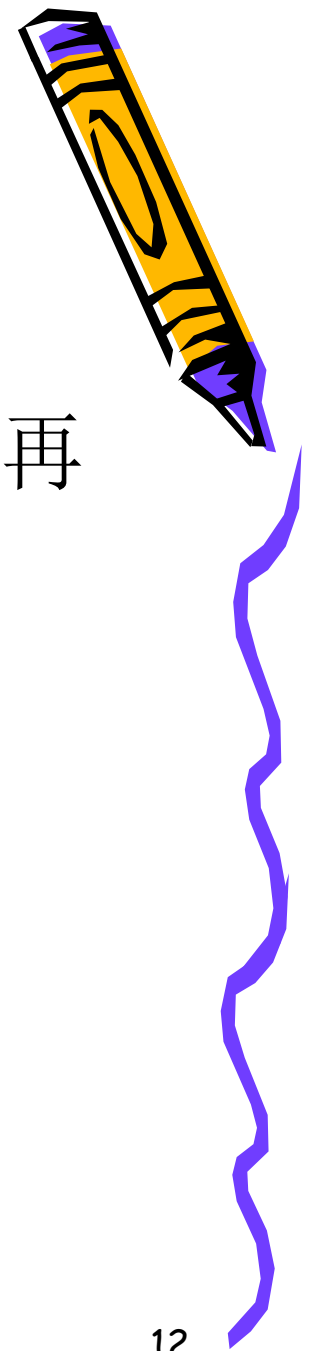


- 程序并发执行为何会失去封闭性和可再现性?
 - 共享资源、资源状态由多个程序更改，一个程序的运行将受到另一个程序运行的影响，失去封闭性；
 - 这将使得计算结果受并发程序执行顺序的影响，也就是说在相同的起始条件下多次运行会产生不同结果，失去可再现性。



习题4

(程序并发执行的特性)



- 程序并发执行为何会失去封闭性和可再现性?

进程A:

While(true)

N=N+1

进程B

While(true)

Print(N)

N=0



问题5：进程控制原语



- 常用的进程控制原语有哪些？各完成什么操作？
 - 进程创建原语
 - 进程终止原语
 - 进程阻塞原语
 - 进程唤醒原语
 - 进程挂起原语
 - 进程激活原语

原语：执行期间不能被打断的指令序列



问题6：线程两种基本类型



- 用户级线程和核心级线程有何区别？

① 线程的调度与切换时间

用户级线程的切换通常发生在一个应用进程的多个线程之间，无须通过OS的内核进行中断，且切换规则也简单，因此其切换速度特别快。而核心级线程的切换时间相对比较慢。



问题6：线程两种基本类型



- 用户级线程和核心级线程有何区别？

① 线程的调度与切换时间

② 从系统调用的角度看

用户级线程调用系统调用时，内核不知道用户级线程的存在，只是当作是整个进程行为，使进程等待并调度另一个进程执行，在内核完成系统调用而返回时，进程才能继续执行。

核心级线程则以线程为单位进行调度，当线程调度系统调用时，内核将其作为线程的行为，因此阻塞该线程，可以调度该进程中的其他线程执行。



问题6：线程两种基本类型



- 用户级线程和核心级线程有何区别？

- ① 线程的调度与切换时间
- ② 从系统调用的角度看
- ③ 从线程执行时间角度看

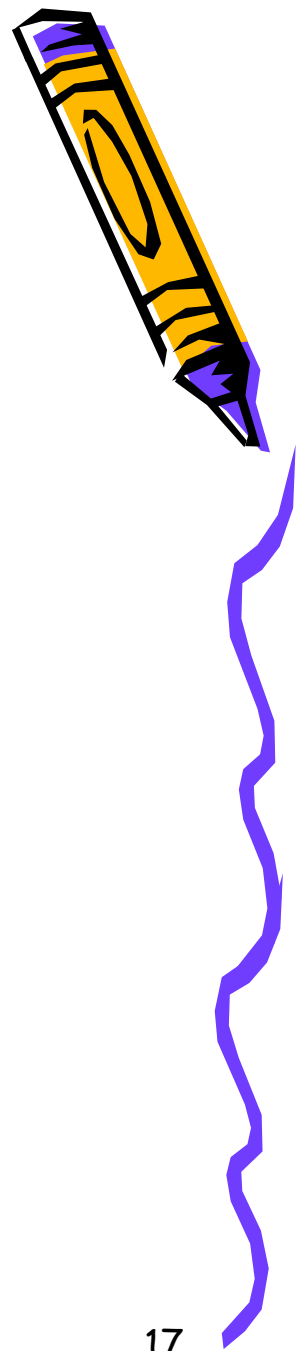
如果用户设置了用户级线程，系统调用是以进程为单位进行的，但随着进程中线程数目的增加，每个线程得到的执行时间就少。

而如果设置的是核心级线程，则调度以线程为单位，因此可以获得良好的执行时间。



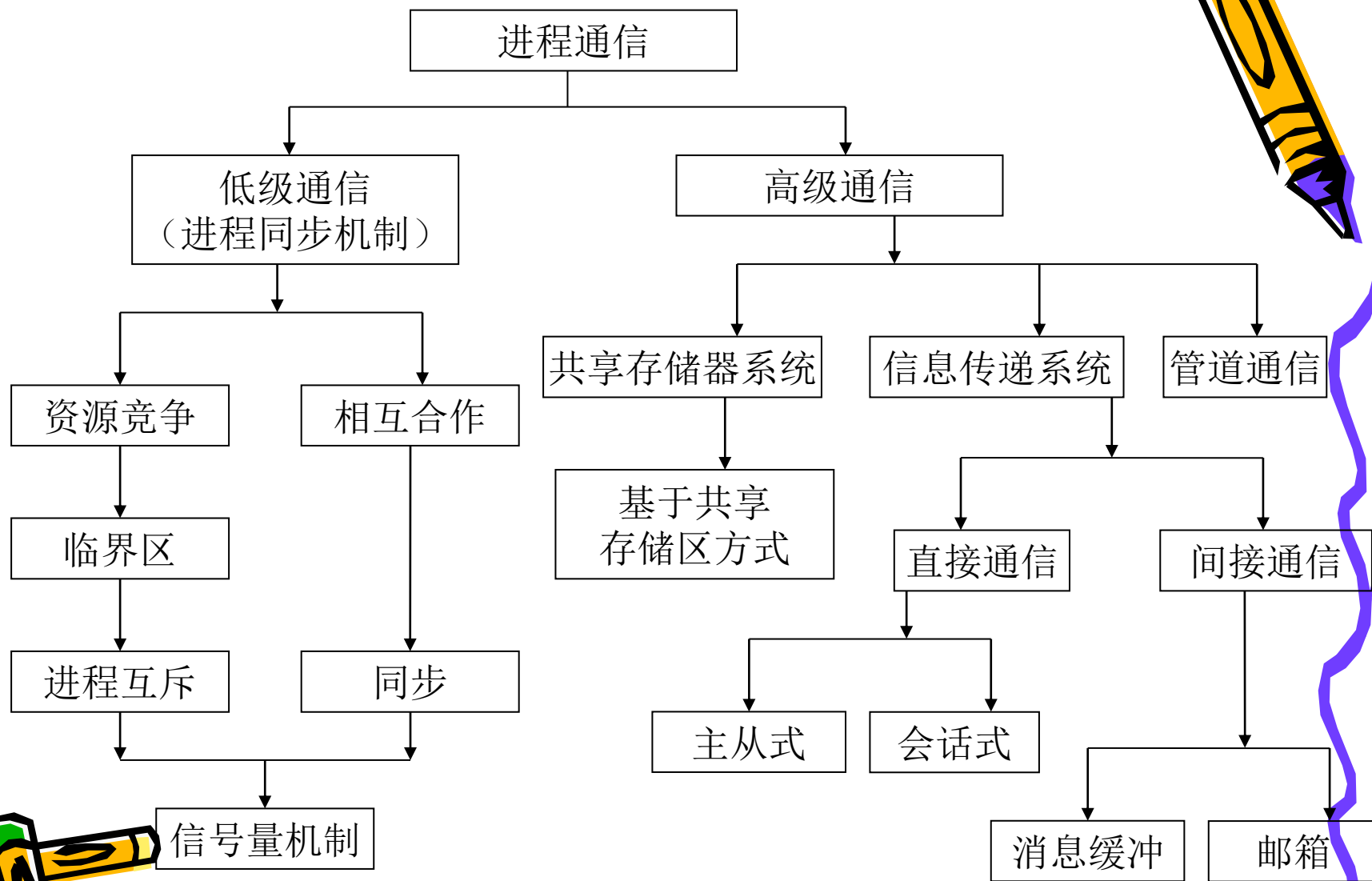
内容提要

- 进程的描述和控制
- 进程的同步与通信
- 调度与死锁



2021-04-22

进程的同步与通信



问题7：临界区



- 处于临界区中的代码的执行是不可中断的吗？
 - 错。进程进入临界区，标明进程正在访问某个临界资源，即不允许其他进程进入访问同一临界资源的临界区。但该进程在临界资源的访问过程中，如正在使用打印机，此时，它可能由于等待打印的完成而处于阻塞状态，因此系统可以调度另一个进程执行，也就是说，处于临界区的代码执行是可以中断的。



问题8：信号量



- 有20个进程共享一个需要保护的区域，每次最多允许5个进程进入这一区域，则信号量的变化范围是（？）

- A、0到5
- B、-15到5
- C、-19到1
- D、-1到5

- 开始时 $S=5$ (表示可用资源数)，假定极限情况下20个进程同时要对临界区进行操作，则 $(5-20) = -15$ 。

- 答案B

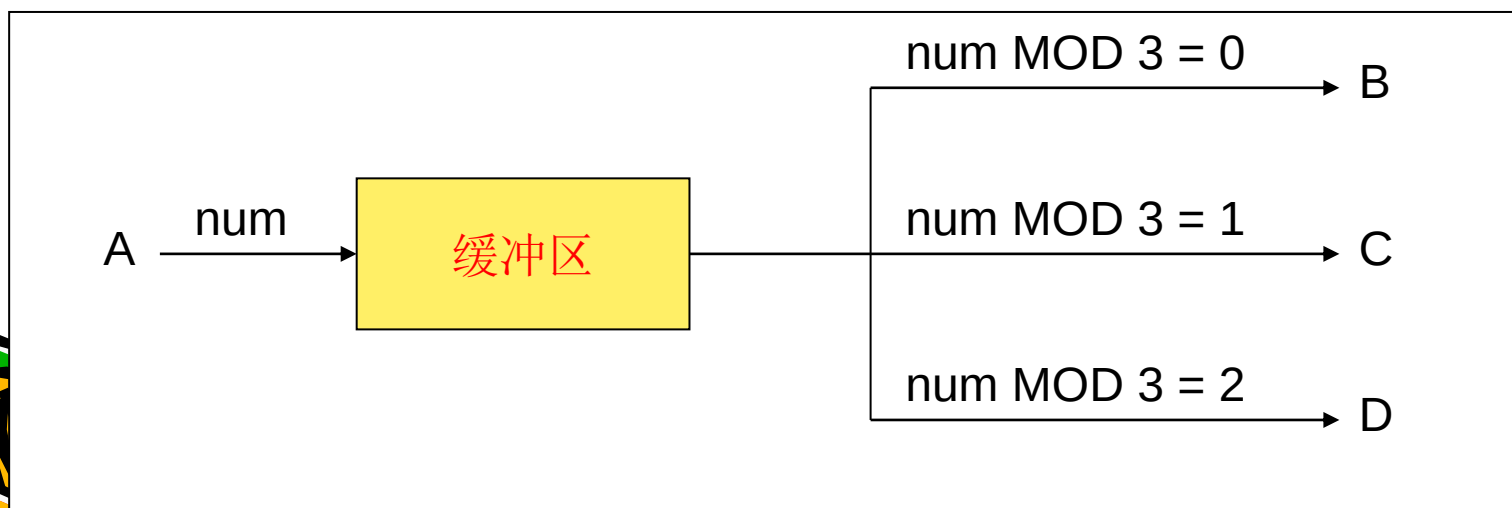
2021-04-22



习题9

- 设有4个进程A、B、C、D共享一个缓冲区(大小为1),
 - 进程A负责循环地从文件读一个整数并放入缓冲区
 - 进程B从缓冲区循环读入MOD 3为0的整数并累计求和
 - 进程C从缓冲区循环地读入MOD 3为1的整数并累计求和
 - 进程D从缓冲区中循环地读入MOD 3为2的整数并累计求和。
- 请用P、V操作写出能够正确执行的程序。

问题描述:



习题9



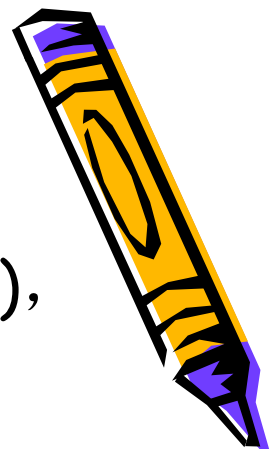
- 设有4个进程A、B、C、D共享一个缓冲区(大小为1),
 - 进程A负责循环地从文件读一个整数并放入缓冲区
 - 进程B从缓冲区循环读入MOD 3为0的整数并累计求和
 - 进程C从缓冲区循环地读入MOD 3为1的整数并累计求和
 - 进程D从缓冲区中循环地读入MOD 3为2的整数并累计求和。
- 请用P、V操作写出能够正确执行的程序。

问题分析：这是一个简单的生产者消费者问题。

所需同步信号量：？



习题9



- 设有4个进程A、B、C、D共享一个缓冲区(大小为1),
 - 进程A负责循环地从文件读一个整数并放入缓冲区
 - 进程B从缓冲区循环读入MOD 3为0的整数并累计求和
 - 进程C从缓冲区循环地读入MOD 3为1的整数并累计求和
 - 进程D从缓冲区中循环地读入MOD 3为2的整数并累计求和。
- 请用P、V操作写出能够正确执行的程序。

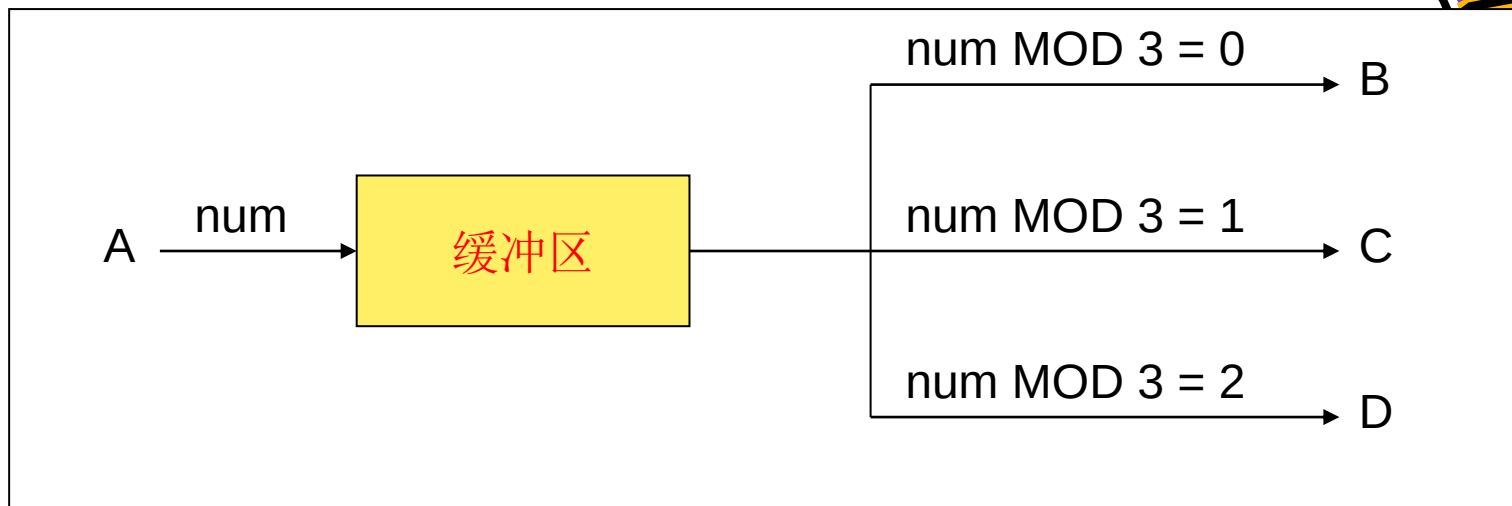
问题分析：这是一个简单的生产者消费者问题。

Empty := 1

SB、SC、SD := 0



2021-04-22



Process PA
Begin
 P(Sempty);
 <写入num至缓冲区>
 if (num MOD 3 = 0)
 V(SB);
 if (num MOD 3 = 1)
 V(SC);
 else
 V(SD);
end

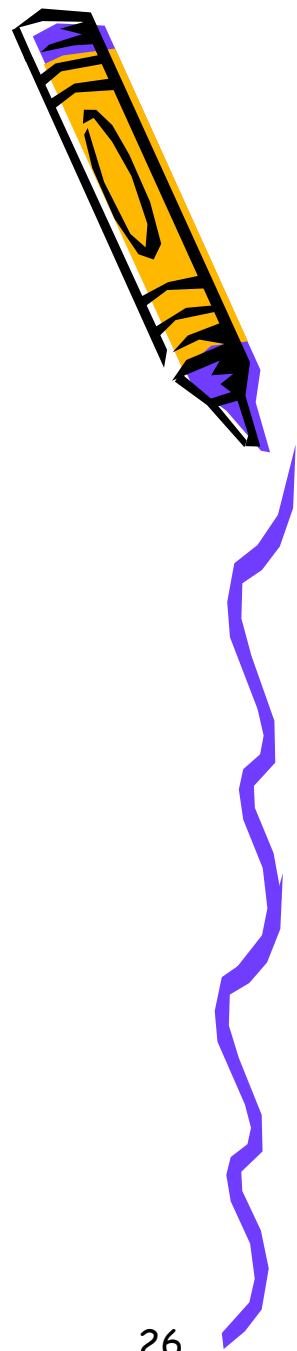
Process PB
Begin
 P(SB);
 <从缓冲区读入num并累计求和>
 V(Sempty);
end

Process PC
Begin
 P(SC);
 <从缓冲区读入num并累计求和>
 V(Sempty);
end

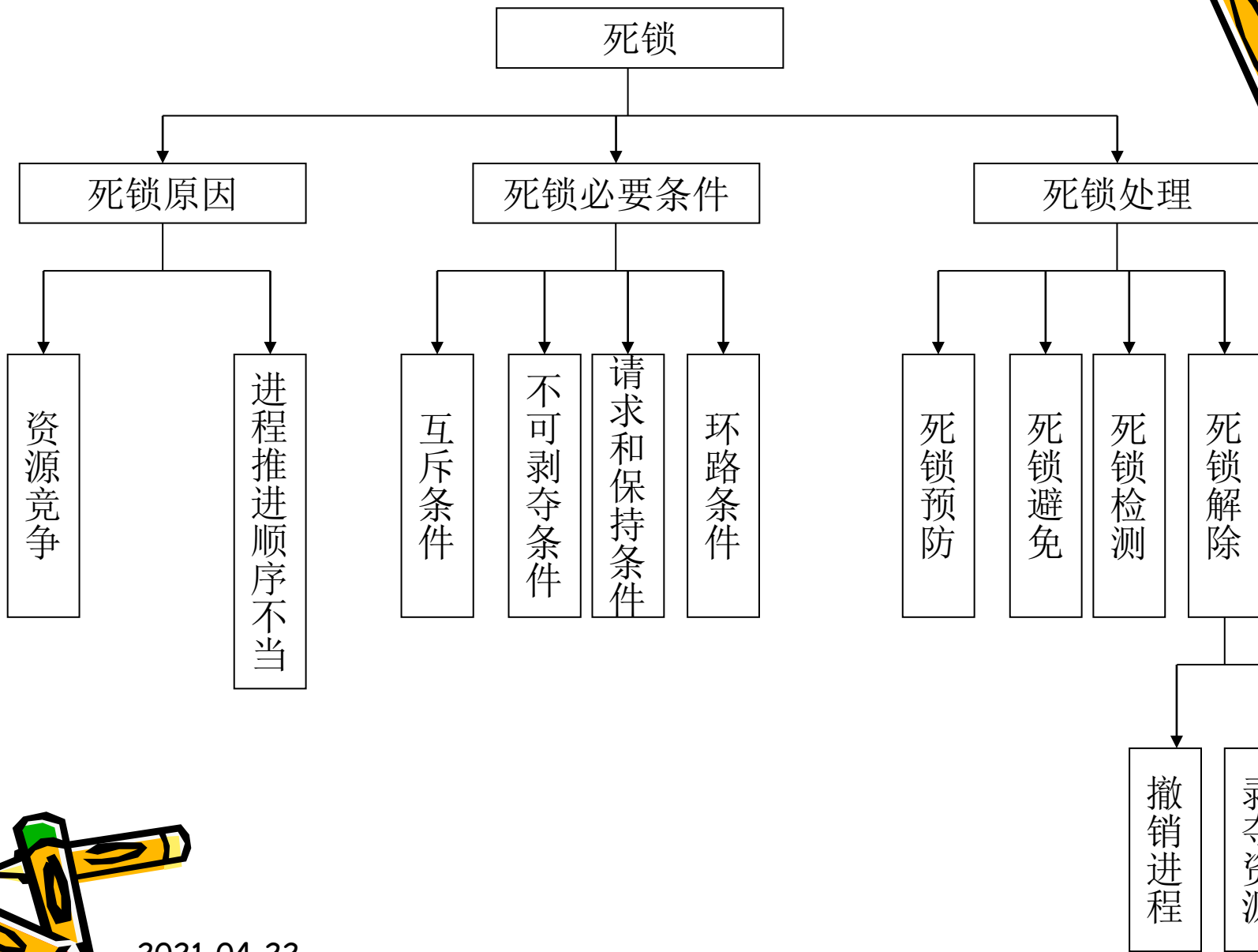
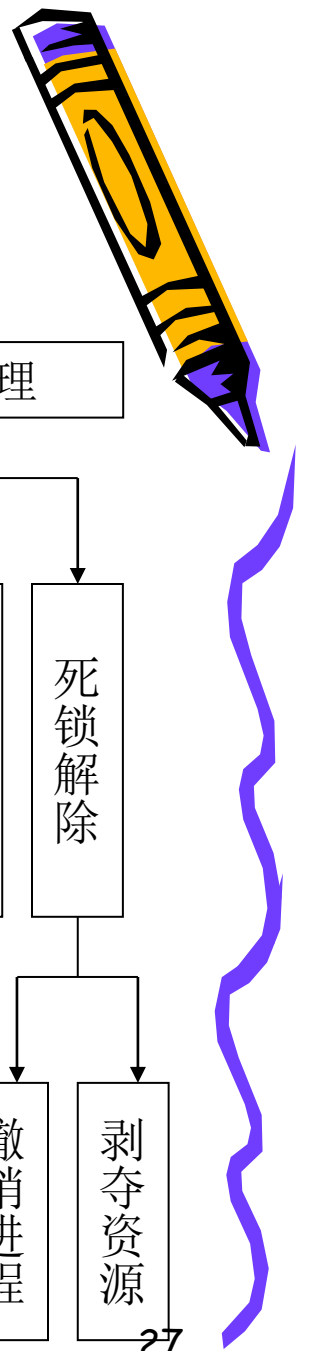
Process PD
Begin
 P(SD);
 <从缓冲区读入num并累计求和>
 V(Sempty);
end

内容提要

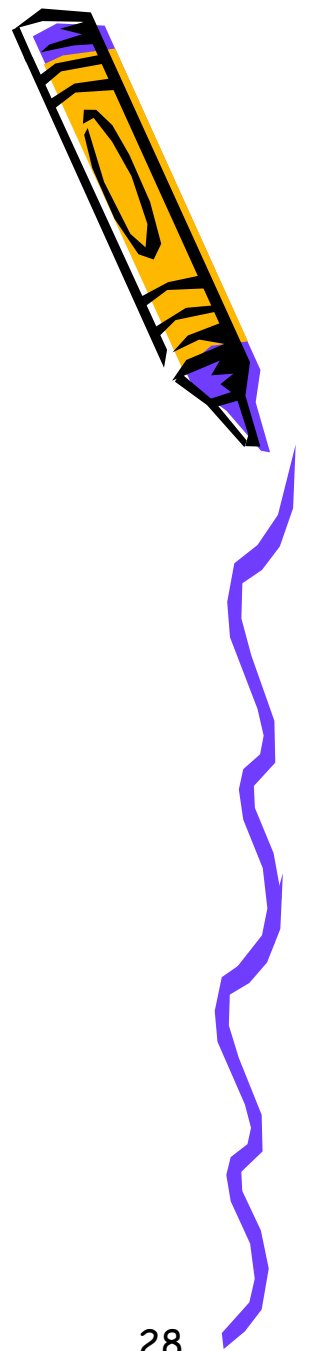
- 进程的描述和控制
- 进程的同步与通信
- 调度与死锁



2021-04-22



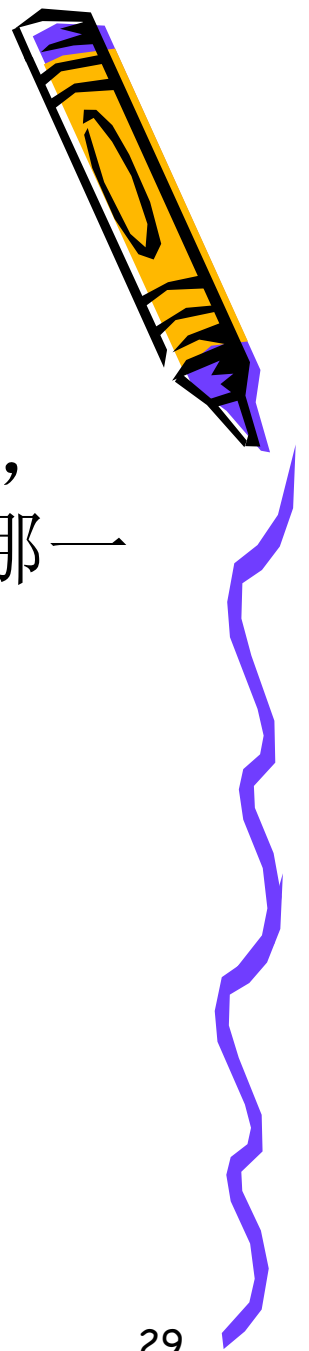
问题10：死锁



- 产生死锁的基本原因是（ ）和（ ）。
- 答案：竞争资源和进程推进顺序不当
- 死锁避免是根据（ ）采取措施实现的。
 - A、配置足够的系统资源
 - B、使进程的推进顺序合理
 - C、破坏死锁的4个必要条件之一
 - D、防止系统进入不安全状态
 - 答案D



问题11：死锁



- 当采用动态资源分配方法预防死锁时，破坏了产生死锁的4个必要条件中的哪一个？
- 答案：部分分配（请求与保持）



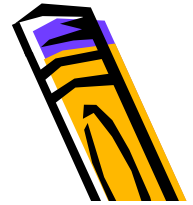
问题12：死锁



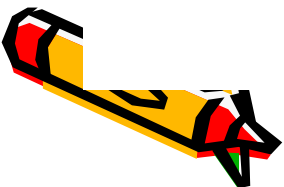
- 某系统中只有**11**台打印机，**N**个进程共享打印机，每个进程要求**3**台，当**N**取值不超过（ ）时，系统不会发生死锁？
- 最坏情况下，**N**个进程每个都得到**2**台打印机，都去申请第**3**台，为了保证不死锁，此时打印机的剩余数目至少为**1**台，则：
 - $11 - 2N \geq 1$
 - $N \leq 5$

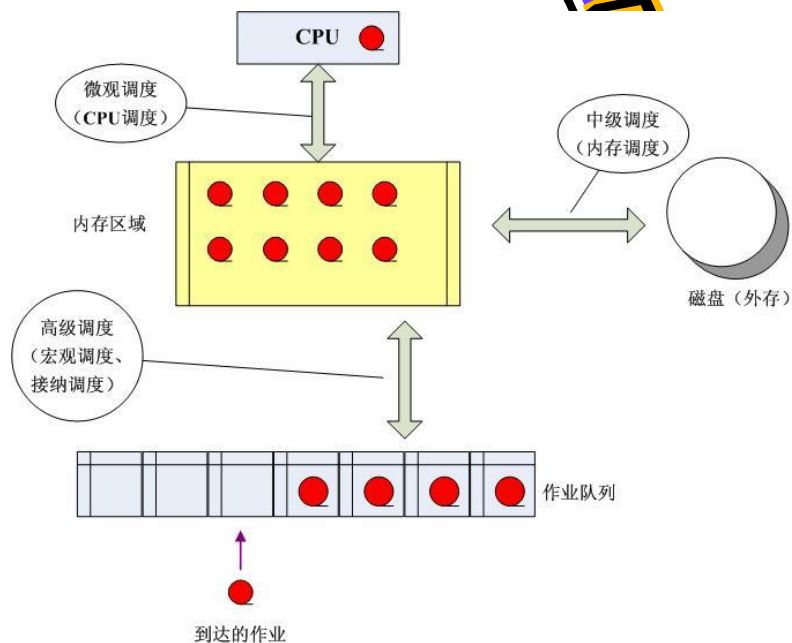
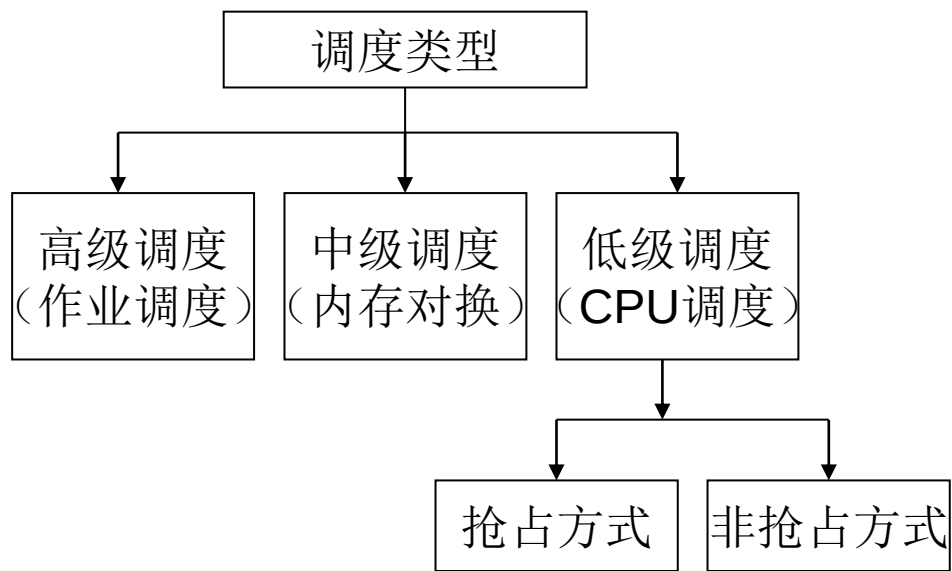


问题13：死锁



- 设系统中仅有一个资源类，其中共有**3**个资源实例，使用此类资源的进程共有**3**个，每个进程至少请求一个资源，它们所需资源最大量的总和为**X**，则发生死锁的必要条件是（**X**的取值）：（？）
- 假设**3**个进程所需该类资源数分别是**a,b,c**个，因此有：
 - $a+b+c = X$
 - 假设发生了死锁，也即当每个进程都申请了部分资源，还需最后一个资源，而此时系统中已经没有了剩余资源，即：
 - $(a-1)+(b-1)+(c-1) \geq 3$
 - $X = a+b+c \geq 6$
 - 因此，如果发生死锁，则必须满足的必要条件是（ **$X \geq 6$** ）



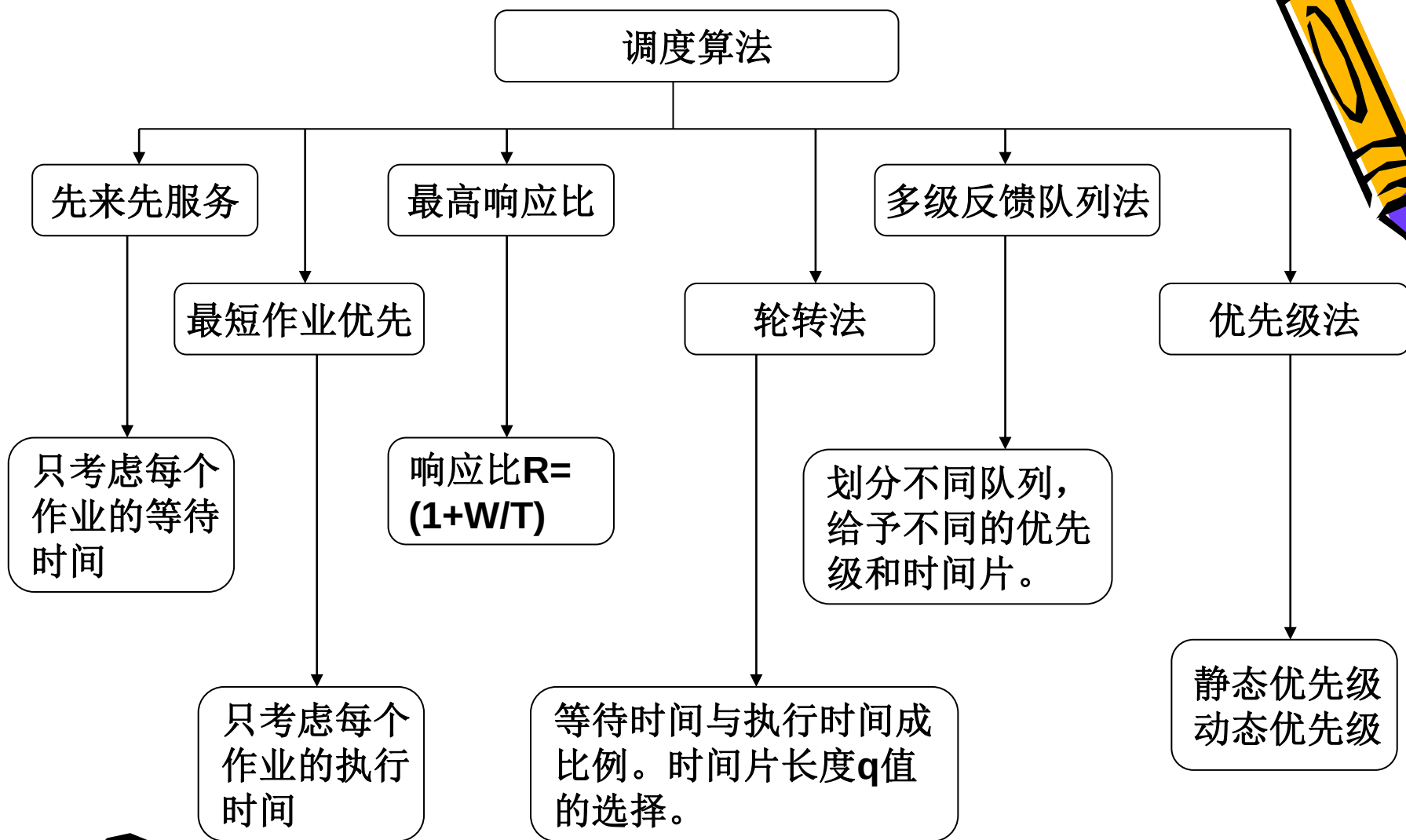
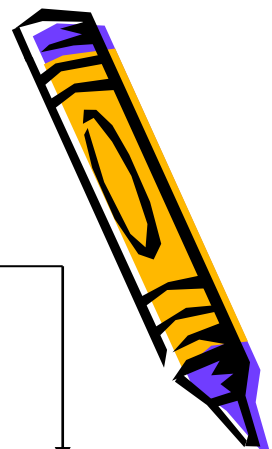


高级调度的目标是载入合理数目的计算型作业和I/O型作业，以使得CPU利用率和外设利用率都尽可能高。

中级调度主要完成虚拟内存管理相关的换入换出操作。

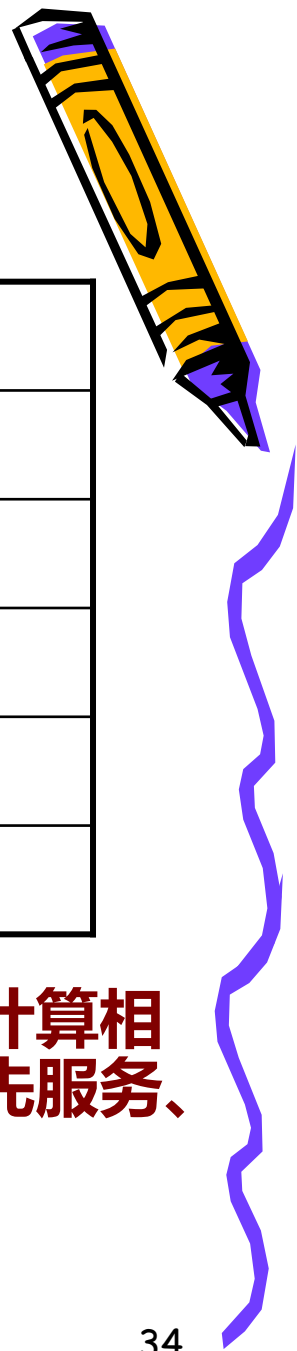
低级调度主要是完成CPU的分配工作，以尽可能保持CPU处于工作状态。





性能衡量指标：周转时间、带权周转时间、响应时间

问题14：调度算法

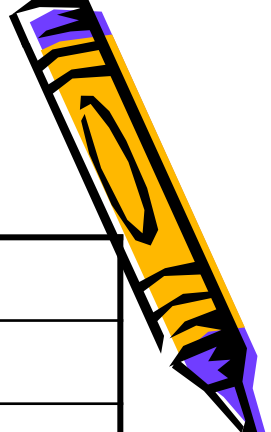


进程	就绪时间	服务时间
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

- 根据不同的调度算法，给出相应的调度过程，并计算相关的平均周转时间和平均带权周转时间。（先来先服务、最短作业优先、轮转法、高响应比优先算法）



调度算法



进程	就绪时间	服务时间
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

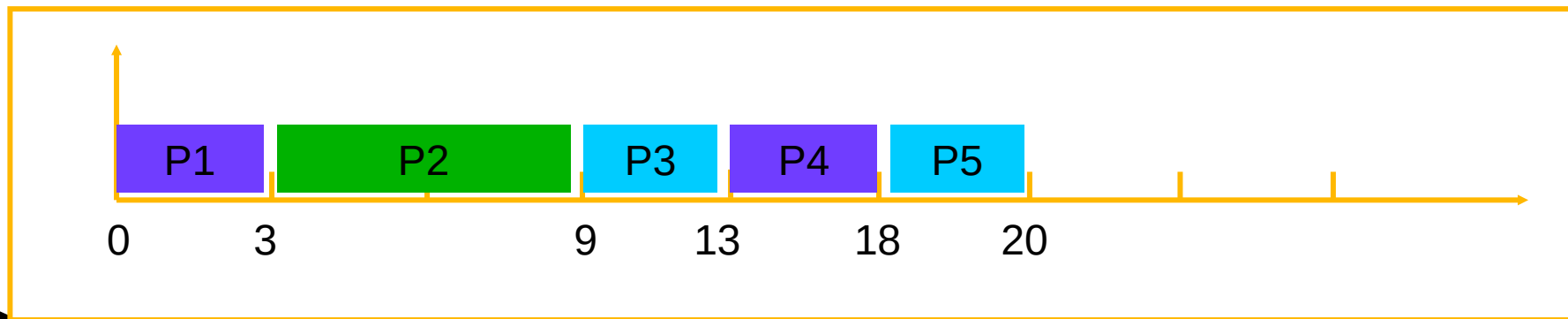
关键点：系统的队列调度模型，
即进程怎么进入就绪队列，以及如何从就绪队列中选择一个进程投入运行。



调度算法

进程	就绪时间	服务时间
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

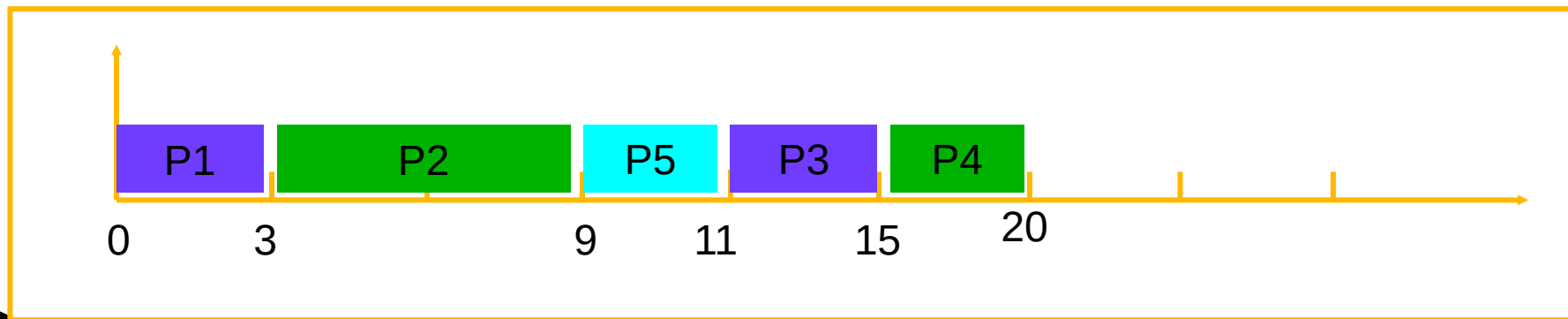
FCFS 调度算法调度顺序图



调度算法

进程	就绪时间	服务时间
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

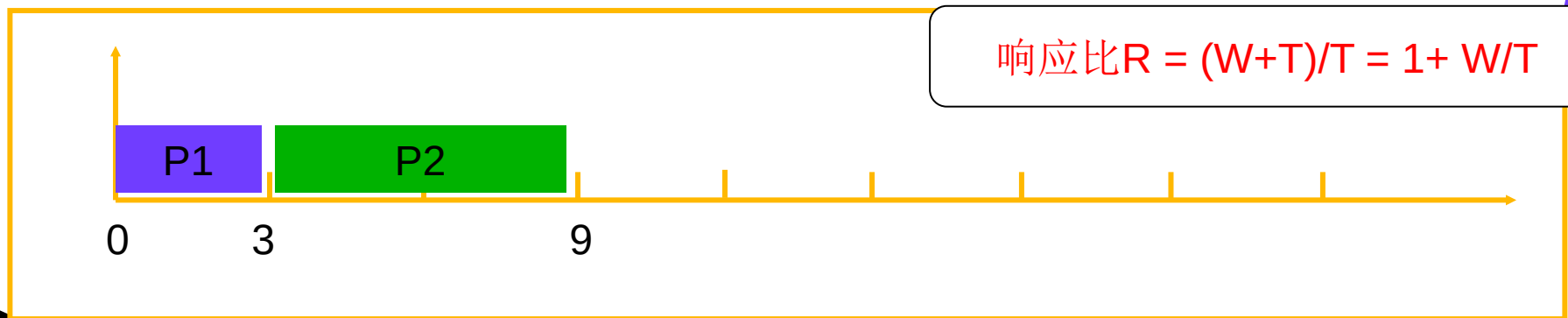
SJF 调度算法调度顺序图

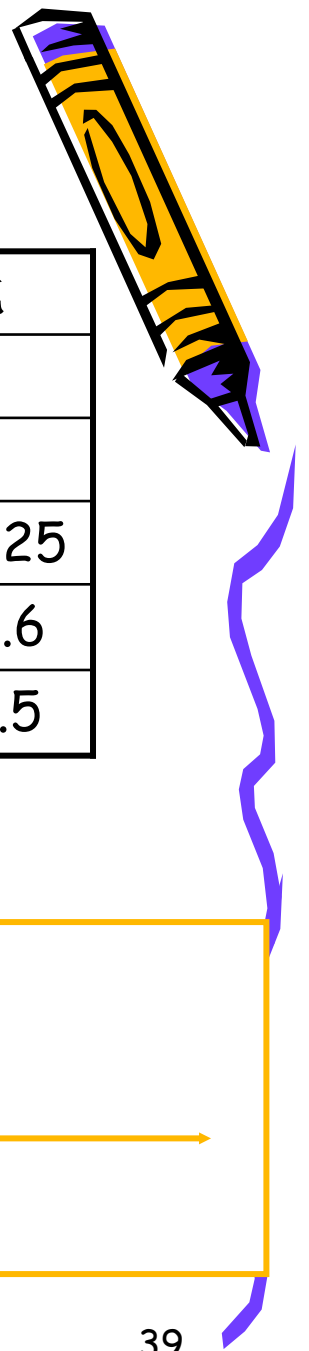


调度算法

进程	就绪时间	服务时间
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

HRRN 调度算法调度顺序图



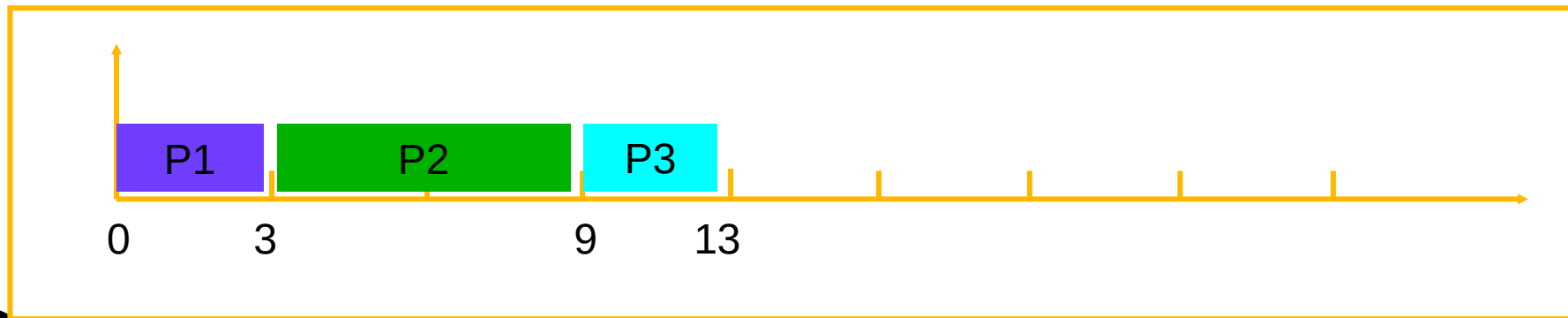


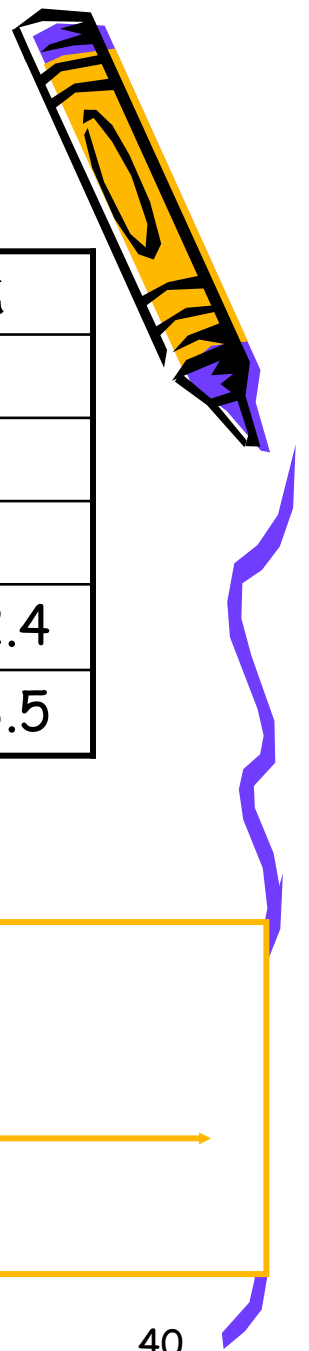
时刻T=9

调度算法

进程	就绪时间	服务时间	状态	等待时间	响应比
P1	0	3	F		
P2	2	6	F		
P3	4	4	W	$9-4=5$	$1+5/4=2.25$
P4	6	5	W	$9-6=3$	$1+3/5=1.6$
P5	8	2	W	$9-8=1$	$1+1/2=1.5$

HRRN 调度算法调度顺序图



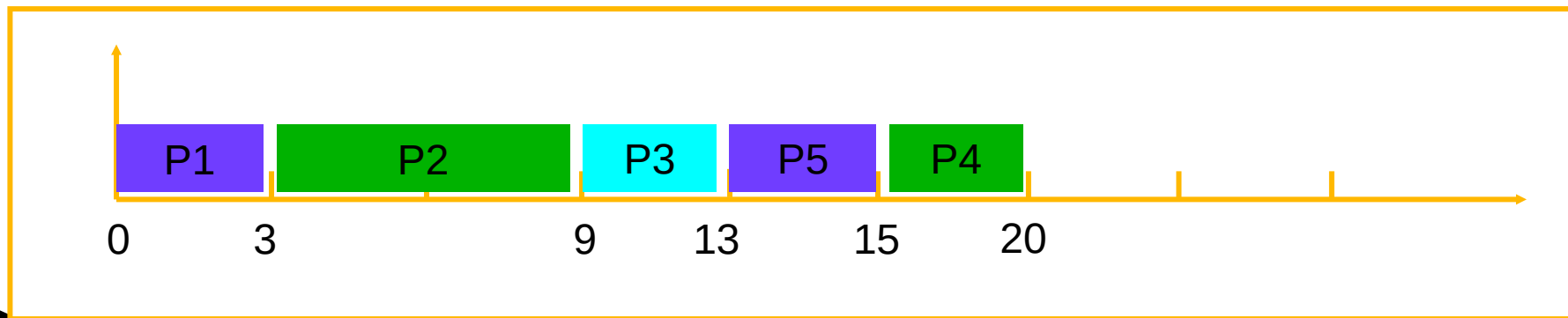


调度算法

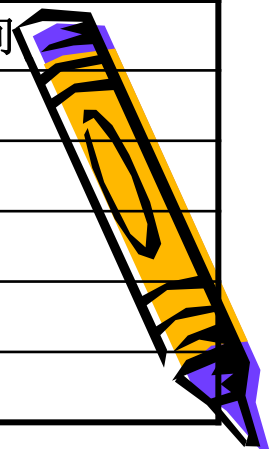
时刻T=13

进程	就绪时间	服务时间	状态	等待时间	响应比
P1	0	3	F		
P2	2	6	F		
P3	4	4	F		
P4	6	5	W	$13-6=7$	$1+7/5=2.4$
P5	8	2	W	$13-8=5$	$1+5/2=3.5$

HRN 调度算法调度顺序图

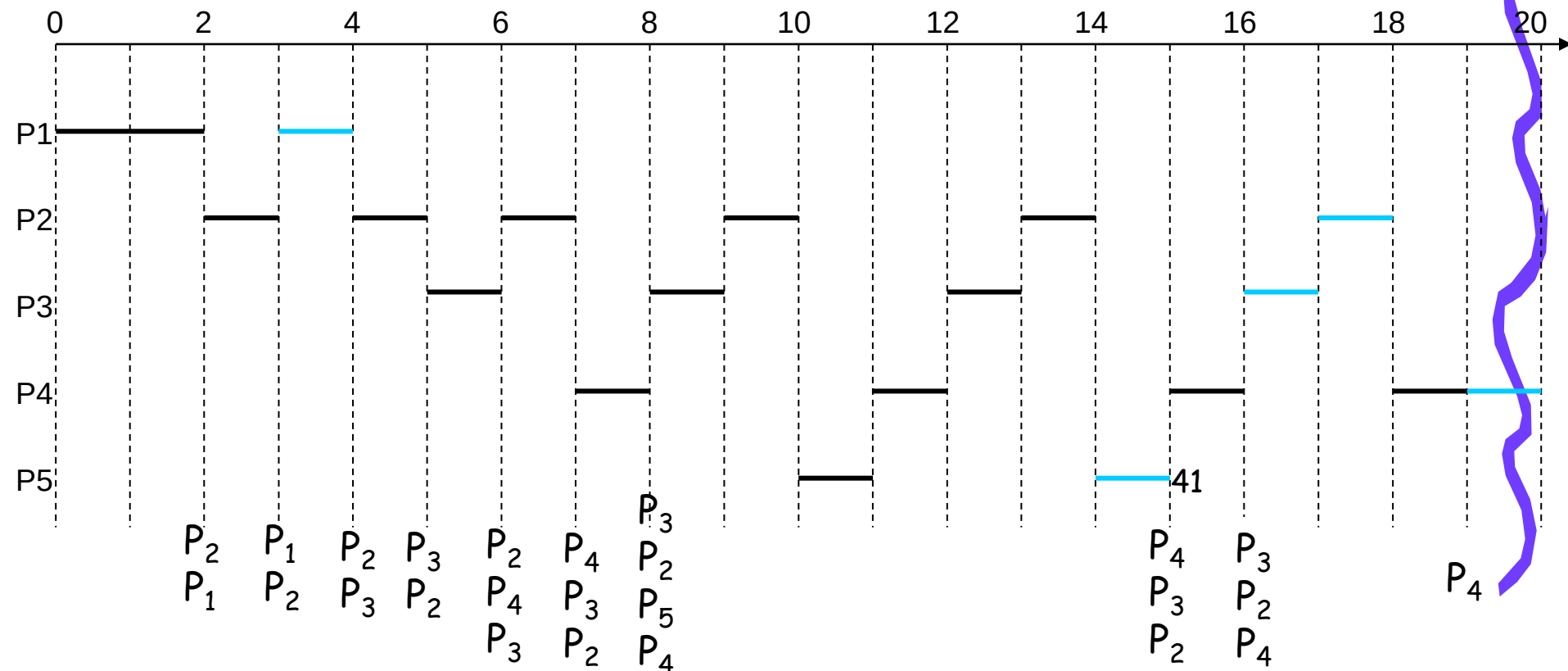


进程	就绪时间	服务时间
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2



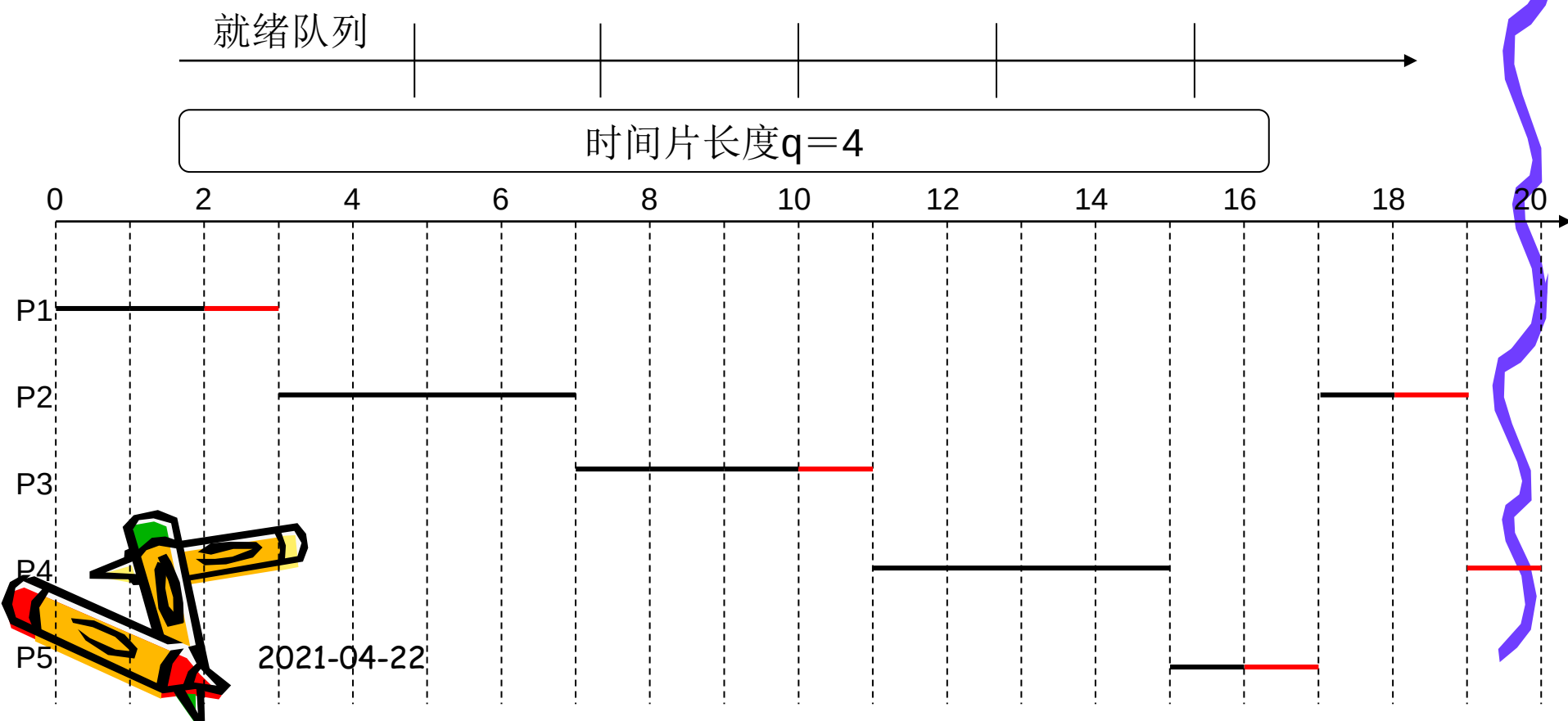
轮转法RR调度顺序

时间片长度 $q = 1$ 和 $q = 4$ 和 $q = 6$



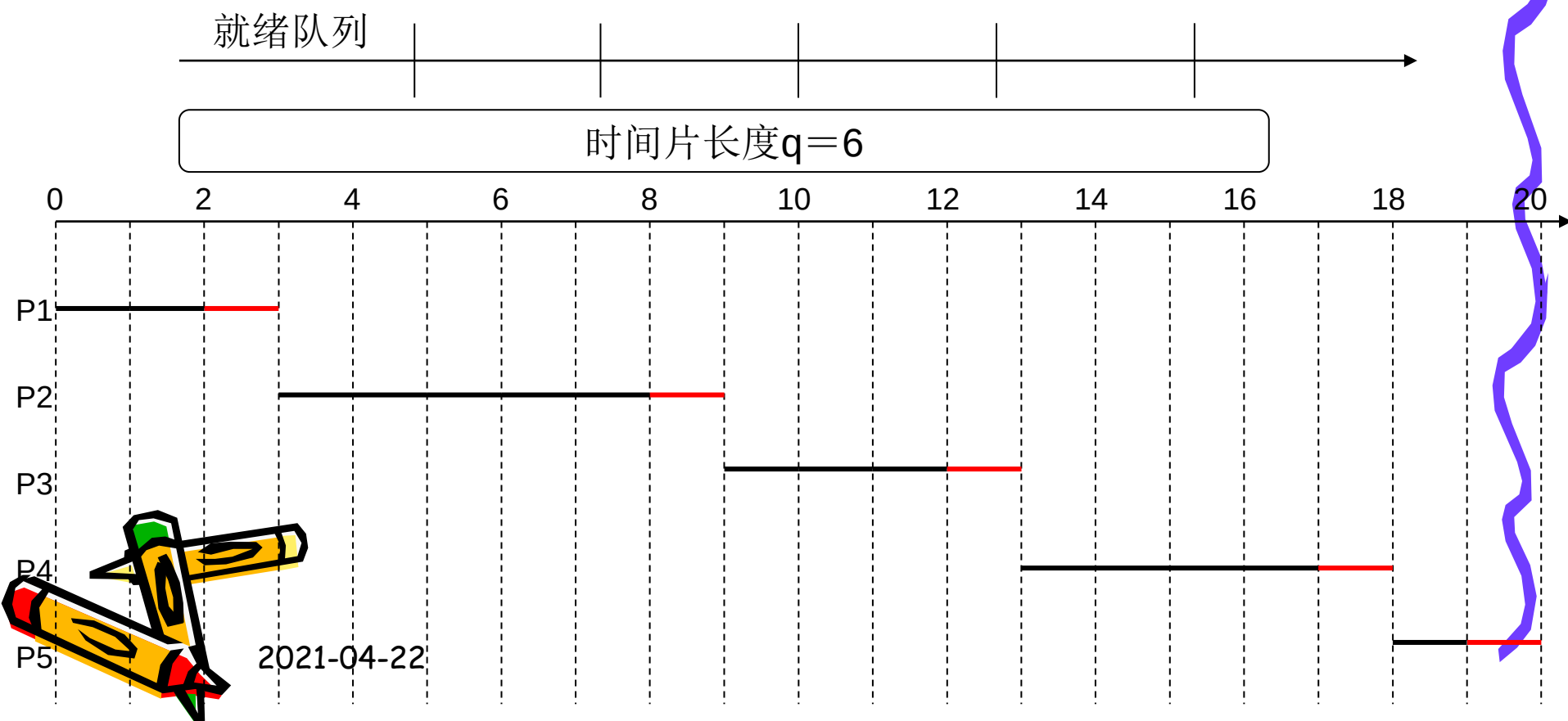
进程	就绪时间	服务时间
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

轮转法RR调度顺序



进程	就绪时间	服务时间
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

轮转法RR调度顺序



• 综合应用题

1. 三个进程P1、P2、P3互斥使用一个包含N ($N > 0$) 个单元的缓冲池。P1每次用produce()生成一个正整数并用put()送入缓冲区某一空单元中；P2每次用getodd()从该缓冲区中取出一个奇数并用countodd()统计奇数个数；P3每次用geteven()从该缓冲区中取出一个偶数并用counteven()统计偶数个数。请用信号量机制实现这三个进程的同步与互斥活动，并说明所定义信号量的含义。要求用伪代码描述。

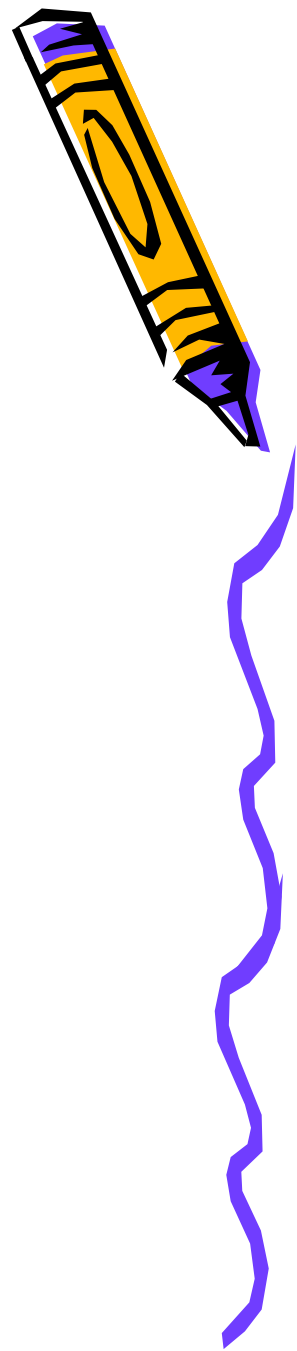




- 生产者—消费者问题改进：
 - 这里的变化主要在于我们把产品分成了两类，这样就有两类消费者；生产者每生产一件产品，需要判别属于哪一类，根据产品的类别不同向不同的消费者发送同步信息。消费者根据产品的有无来决定是否继续消费。
- 1. 信号量定义：
 - 互斥信号量**mutex**: 实现3个进程对缓冲池的互斥使用；
 - 资源信号量**empty**: 表示缓冲池中空缓冲区的数量; (P1, P2, P3)
 - 资源信号量**odd**:表示奇数满缓冲区的数量; (P1, P2)
 - 资源信号量**even**:表示偶数满缓冲区的数量; (P1, P3)



```
semaphore mutex=1;
semaphore odd=0, even=0;
semaphore empty=N;
main()
cobegin{
Process P1
while(True)
{
    number=produce();
    wait(empty);
    wait(mutex);
    put();
    signal(mutex);
    if number%2==0
        signal(even);
    else
        signal(odd);
}
```



Process P2

while (true)

{

wait(odd);

wait(mutex);

getodd();

signal(mutex);

signal(empty);

countodd();

}

Process P3

while (true)

{

wait(even);

wait(mutex);

geteven();

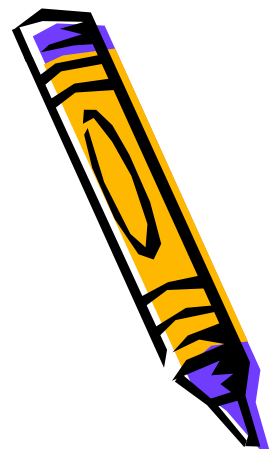
signal(mutex);

signal(empty);

counteven();

}

}coend



①能正确给出互斥信号量定义与含义

②能正确给出3个同步信号量定义与含义

③能正确描述P1、P2和P3进程活动

2. 某机场的入口有**1**条通道，该通道有**3**名证件查验员，再往里是一个安检室，内有**5**名安检员。安检室可容纳五人等待（另有五人正在被检查）。入场者可让任意一名证件查验员检查，在证件查验员核对正确并放行后进入安检室等待并被检验，安检员放行后进入机场。证件查验员无人时休息，来人时被来人唤醒，查验来人证件后，确认安检室有空位后让来人入安检室。安检员无人时休息，直到被来人唤醒，检查完来人后放行来人入机场。

请用信号量机制实现上述过程中的同步和互斥。



- 为唤醒证件查验员和安检员各设一个资源信号量，初值为**0**。来人需互斥地使用**3**个证件查验员，以及**5**个安检员，分别设置一个资源信号量。为放行进入安检室设一个初值为**10**的资源信号量。为每个来人设置**2**个同步用的信号量，以实现放行。

```
semaphore need_cred_check=0, need_safe_check=0;  
semaphore cred_checker=3, safe_checker=5;  
semaphore cred_oki=0, safe_oki=0;  
semaphore room_empty=10;
```



1) 来人的进程:

```
arriver () {
```

```
    signal(need_cred_check); //唤醒证件检查员, 初值为0
```

```
    wait(cred_checker); //申请1个证件检查员, 初值为3
```

```
    被检查证件;
```

```
    wait(cred_oki);
```

```
    signal(cred_checker); //释放1个证件检查员
```

```
    进入安检室
```

```
    signal(need_safe_check); //唤醒1个安全检查员, 初值为0
```

```
    wait(safe_checker); //申请安全检查员, 初值为5
```

```
    被安全检查
```

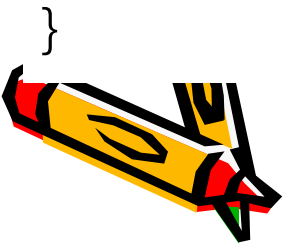
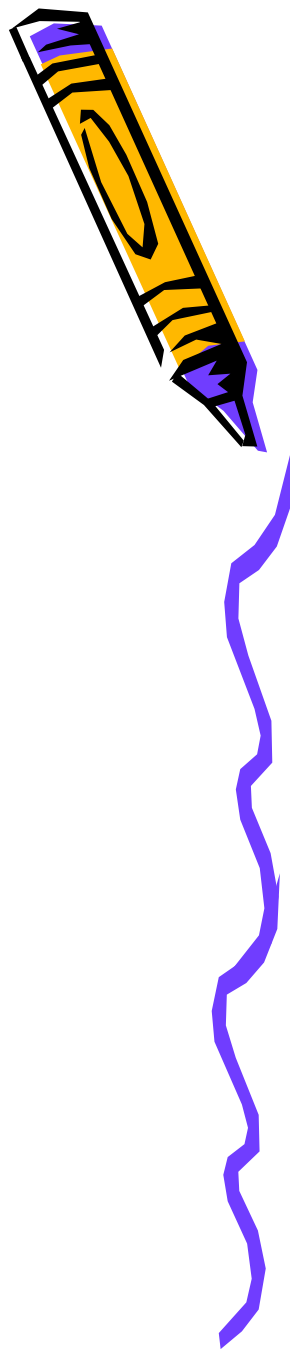
```
    wait(safe_oki)
```

```
    signal(safe_checker); //释放1个安检没员
```

```
    signal(room_empty);
```

```
    进入机场
```

```
}
```

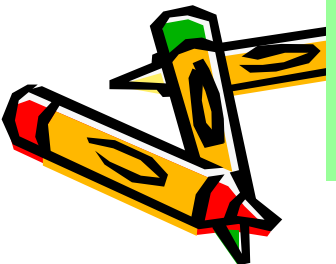
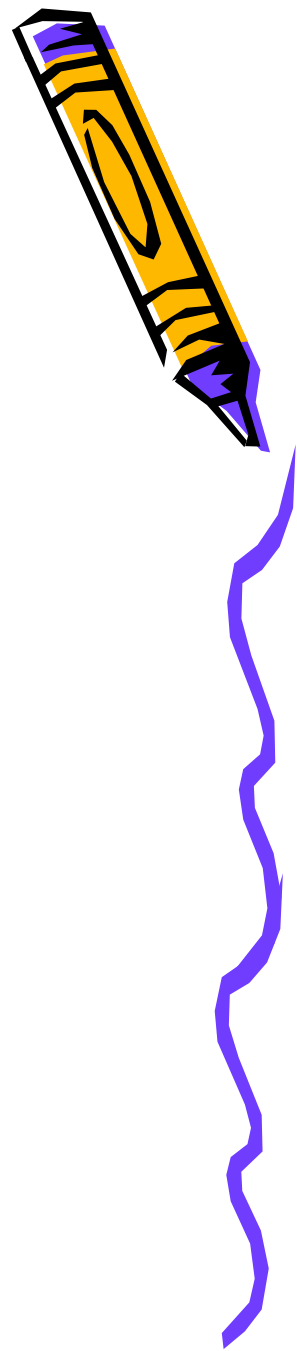


(2) 证件查验员的进程:

```
cred_checker(){  
while(1){  
wait(need_cred_check);  
检查进入者的证件  
wait(room_empty);  
signal(cred_oki);  
}}
```

(3) 安检员进程

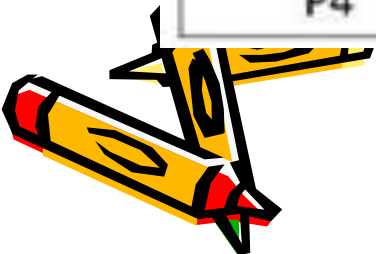
```
safe_checker(){  
while(1){  
wait(need_safe_check);  
检查一个进入者的安全  
signal(safe_oki);  
}  
}
```





3. 假设5个进程P0, P1, P2, P3, P4共享三类资源R1, R2, R3, 这些资源总数分别是18, 6, 22. T0时刻的资源分配情况如下表所示, 此时是否存在一个安全序列? 如果存在, 该安全序列是什么? 请给出解题步骤

进程	已分配资源			资源最大需求		
	R1	R2	R3	R1	R2	R3
P0	3	2	3	5	5	10
P1	4	0	3	5	3	6
P2	4	0	5	4	0	11
P3	2	0	4	4	2	5
P4	3	1	4	4	2	4

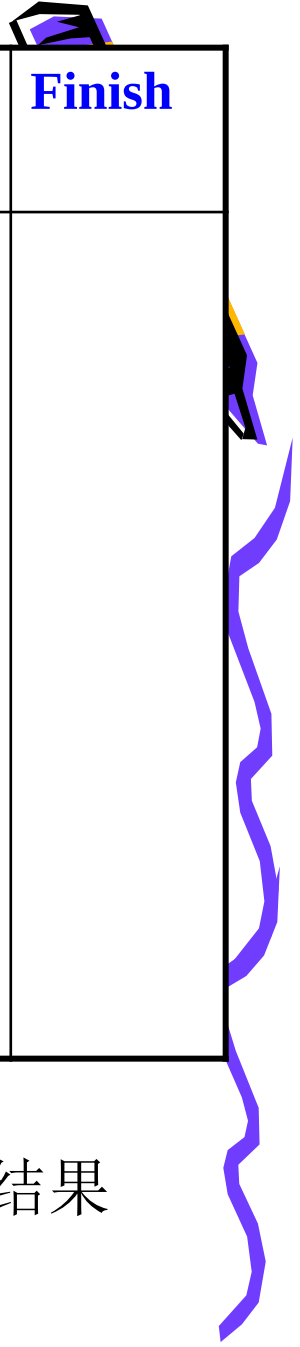


(1)判断 T_0 时刻的安全性

- 判断 T_0 时刻是否是安全状态?

	Max			Allocation			Need			Available		
	R ₁	R ₂	R ₃	R ₁	R ₂	R ₃	R ₁	R ₂	R ₃	R ₁	R ₂	R ₃
P0	5	5	10	3	2	3	2	3	7	2	3	3
P1	5	3	6	4	0	3	1	3	3			
P2	4	0	11	4	0	5	0	0	6			
P3	4	2	5	2	0	4	2	2	1			
P4	4	2	4	3	1	4	1	1	0			

利用安全性算法检测。



	Work	Need	Allocation	Work+ Allocation	Finish

存在一个安全序列

能给出正确的解题步骤及其结果

{P3,P4,P2,P1,P0}

{P1,P2,P3,P4,P0}

